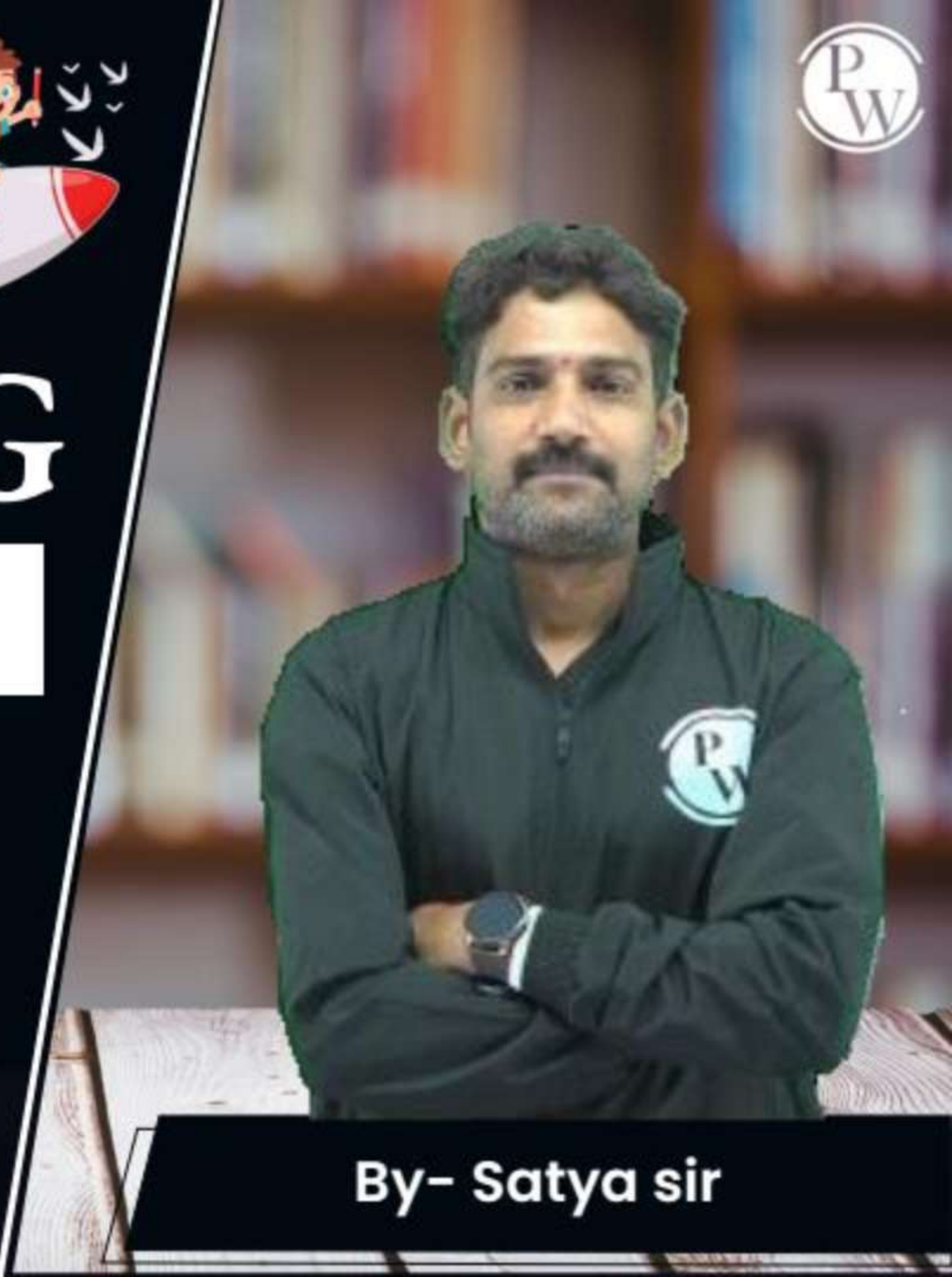
CS & IT ENGINEERING

'C' Programming

Structures, Unions

One Shot



By- Satya sir

Recap of Previous Lecture

- Tree Recursion
- Nested Recursion
- Indirect Recursion
- Storage classes
  - auto
  - register
  - Static

Topics to be Covered

- extern storage class
- Examples
- Structures and Unions

Topic : Storage Classes

extern Storage class

- A Variable is declared External to the current block (or) Current Program.
- extern datatype Variable;

Ex:

```
int i = 5 // 10 ;
Void f1( )
{   int j = 7;
    i = i + j; // i = 10
}
Void f2( )
{   int k = 3;
    k = i++; // k = 10
             // i = 11
}
Void main( )
{
    i = i - 2; // i = 3
    Printf("%d", i);  // 3
    f1( );
    f2( );
}
```

Topic : Storage Classes

Ex:

Sample.h // User-defined header file

```
int i=5;
```

Same directory

Test.c

```
#include <stdio.h>
#include "Sample.h"
Void f1( )
{
  extern int i;
  int j;
  j=i+2; // j=11
  Printf("%d", j); // 11
}
```

i=9

```
void main( )
{
  extern int i;
  int k=4;
  i=i+k; // i=9
  Printf("%d", i); //9
} f1();
```

Memory is Not allocated

Not allocated

o/p: 9 11

Every global variable is External. unConditionally.

But, Every External variable is Conditionally Global.

Topic : Storage Classes

Ex:

```
int f1(int x)
{
    static int i=2;
    for(int j=1; j<x; j++)
    {
        i = i+j;
    }
    return i;
}
```

```
→ void main( )
{
    int i, result=0;
    for( i=1; i<5; i++)
        result = result + f1(i);
    Printf("%d", result);
}
```

o/p: 23

f1(1)	f1(2)	f1(3)	f1(4)
x=1	x=2	x=3	x=4
1<1 false	j=1, 1<2 True	j=1 i=4	j=1 i=7
return 2	i=3	j=2 i=6	j=2 i=9
	return 3	return 6	j=3 i=12
			return 12

i=1 result = 0 + f1(1) = 0+2 = 2

i=2 result = 2 + f1(2) = 2+3 = 5

i=3 result = 5 + f1(3) = 5+6 = 11

i=4 result = 11 + f1(4) = 11+12 = 23

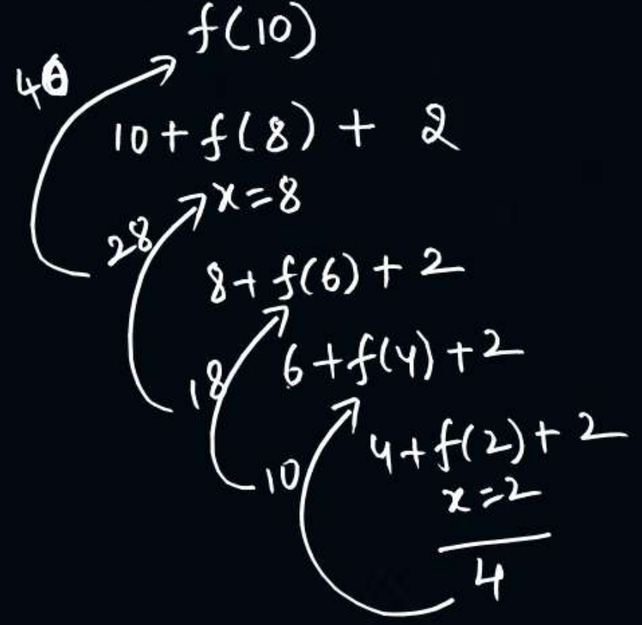
result = 23

Ex: 2

```
int f(int x)
{
    Static int i=2
    if( x <= i)
       return x+i;

    return   x+ f(x-2) +i;
}
→ Void main( )
{
    int result;
    result = f(10);
    Printf("%d", result);
}
```

o/p = 40



Topic : Structures & Unions in Single Shot

Ex:3

```
int fun(int x)
{
    static int i = 2;   // 4 6
    if(x<=i)
        return i;
    i = i+2;
    return x+ fun(x-3)+i;
}
Void main( )
{
    Printf("%d", fun(11));
}
```

o/p: ~~35~~ 37

11+20+4 = 35

fun(11)
i=4
11+fun(8)+4
i=6
8+fun(5)+6
x=5
5<=6 True
return 6
= 8+6+6 = 20

fun(11)
i=2, x=11
i=4
11+fun(8)+ i
i=4, x=8
i=6
8+fun(5)+ i
i=6, x=5
x<=i True
return 6

11+20+6 = 37

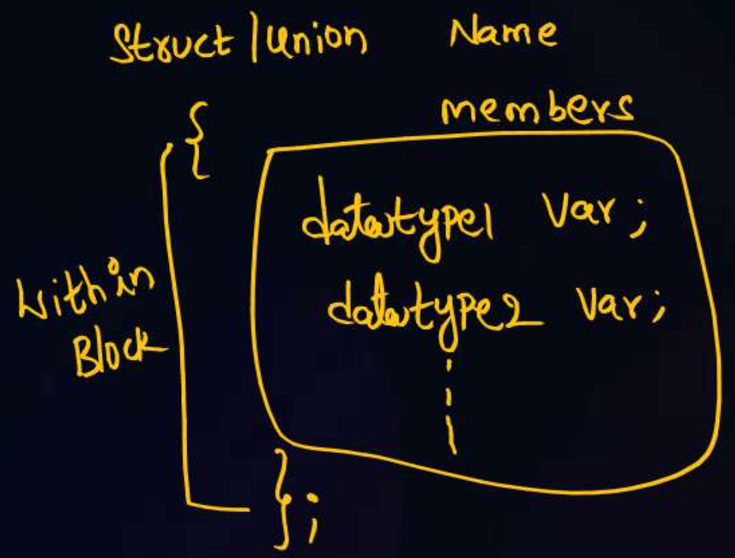
8+6+6=20

Topic : Structures & Unions in Single Shot

Structure, union

- Collection of dissimilar (or) Heterogeneous data

Declaration Syntax



Ex:

```
Struct/union Student_info
{
    char Name[20], address[50];
    int rollno;
    float Percentage;
};
```

NOTE:

- Memory is not directly allocated for any member.
- Memory is allocated on the name of Variables.

Topic : Structures & Unions in Single Shot

Declare Variable for Structure/Union

```
Struct/union  Name
{
    data;
    data 2;
      :
} Variable1, Variable2, *Variable3---;
```

(OR)

```
Struct|union  Name
{
    data member(s);
      :
};
```

```
Void main( )
{
    Struct|union Name Variable(s);
}
```

Ex:

```
union|Struct XYZ
{   int i;
    char j;
    float k;
} V1;
Void main( )
{
    Struct|union XYZ V2;
      ;
}
```

Topic : Structures & Unions in Single Shot

Each Structure Variable memory size = $\sum$ (All members Size)

Each Union Variable memory size = Max (All members Size)

Ex: Let 16-bit Processor

```
Struct  XYZ
{
    int   i;  = 2Bytes
    float j;  = 4Bytes
    char  k;  = 1Byte
} V1, V2;     Σ = 7Bytes
```

V1 (7 Bytes)

V2 (7 Bytes)

- All members Can be accessed Simultaneously.

```
union  XYZ
{
    int   i;  = 2Bytes
    float j;  = 4Bytes
    char  k;  = 1Byte
} V1, V2;     Max() = 4Bytes
```

V1 (4 Bytes)

i, j, k share

V2 (4 Bytes)

i, j, k share

- Only one member Can be accessed at a time

Topic : Structures & Unions in Single Shot

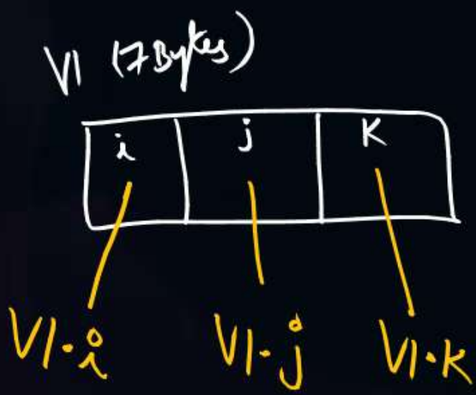
Access members of Structure/union

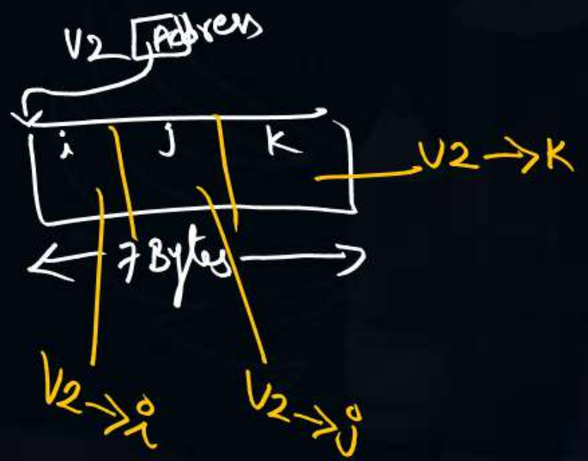
- Through variable only access to members is possible.

If Normal variable	⇒	Variable . member
If Pointer Variable	⇒	Variable → member

Ex:

```
struct/union XYZ
{
    int i;
    float j;
    char k;
} V1, *V2;
```

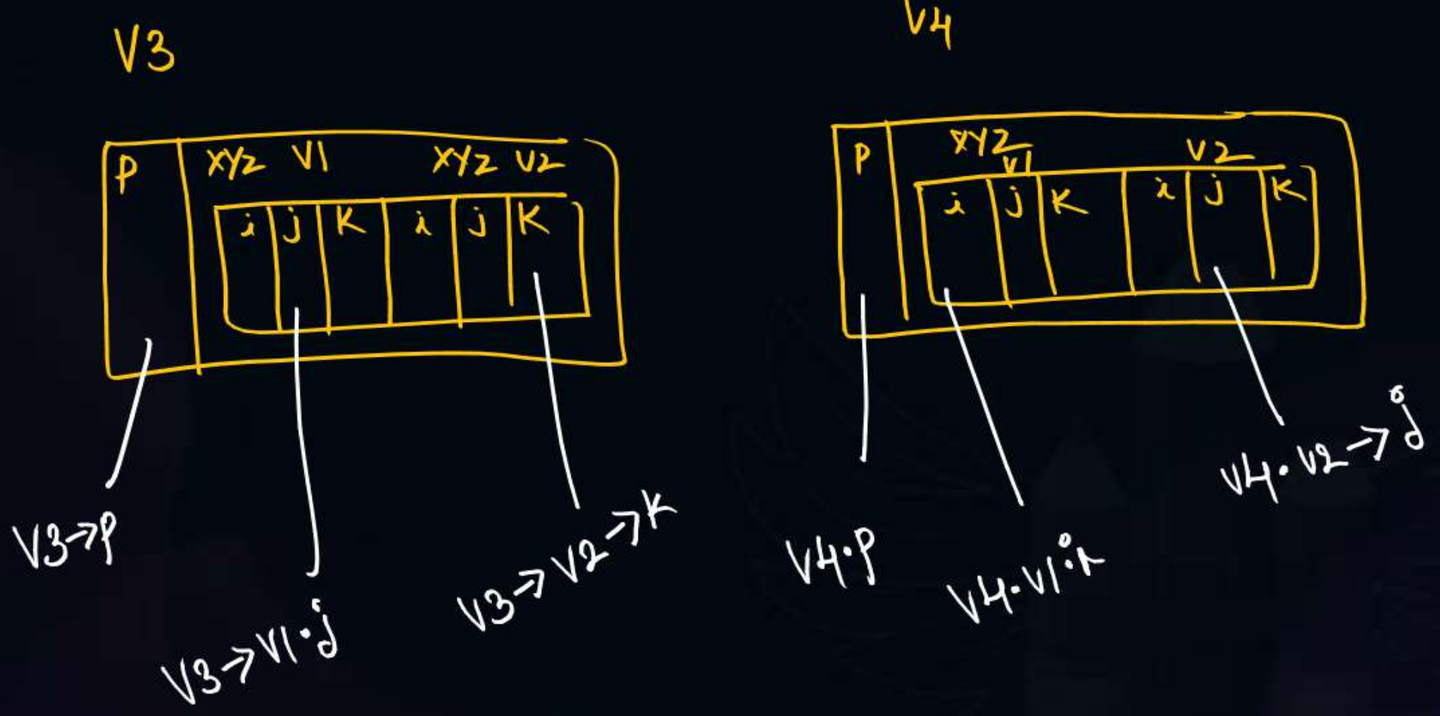





```
Void main()
{
Printf("Enter V1 member details");
Scanf("%d%f%c", &V1.i, &V1.j, &V1.k);
Printf("Enter V2 member details");
Scanf("%d%f%c", &V2→i, &V2→j,
                &V2→k);
}
```

Nested Structure/union

```
union/Struct ABC
{  int P;
union/struct XYZ
     { int i;
       float j;
       char K;
     } V1, *V2;
} *V3, V4;
```



Topic : Structures & Unions in Single Shot

Let 16-bit Processor

Ex:

```
Struct A
{   int i[10];      = 20B
    char j[10];     = 10B
    float k[20];    = 80B
    Struct B ⇒ 16Bytes
    {   int x;      = 2B
        char Y[10]; = 10B
        float z;    = 4B
    } V1;  = 16Bytes
} V2;  20+10+80+16 = 126 Bytes
```

```
Struct A
{   int i[10];      = 20B
    char j[10];     = 10B
    float k[20];    = 80B
    union B = 10B
    {   int x;
        char Y[10];
        float z;
    } V1;  max(2,10,4) = 10B
} V2;  20+10+80+10
```

```
Union A
{   int i[10];      20B
    char j[10];     10B
    float k[20];    80B
    union B         10B
    {   int x;
        char Y[10];
        float z;
    } V1;  max(2,10,4) = 10Bytes
} V2;  max(20,10,80,10) = 80 Bytes
```

```
Union A
{   int i[10];      = 20B
    char j[10];     = 10B
    float k[20];    = 80B
    Struct B = 16Bytes
    {   int x;
        char Y[10];
        float z;
    } V1;  2+10+4 = 16B
} V2;  = max(20,10,80,16) = 80 Bytes
```

sizeof(V2.V1)	: 16 Bytes	10 Bytes	10 Bytes	16 Bytes
sizeof(V2)	: 126 Bytes	120 Bytes	80 Bytes	80 Bytes
sizeof(B)	: 16 Bytes	10 Bytes	10 Bytes	16 Bytes
sizeof(A)	: 126 Bytes	120 Bytes	80 Bytes	80 Bytes

Topic : Structures & Unions - 2

GATE 2000

#Q. Consider the following C declaration

```
struct {
    short s[5];
    union {
    float y;
    long z;
    } u;
}t;
```

Assume that objects of the type short, float and long occupy 2 bytes, 4 bytes and 8 bytes, respectively. The memory requirement for variable t, ignoring alignment considerations, is

A. 22 Bytes
B. 14 Bytes
C. 18 Bytes
D. 10 Bytes

Topic : Structures & Unions - 2

#Hw

#Q. The following C declarations

GATE 2000

```
struct node{
  int i:
  float j;
};
struct node *s[10];
```

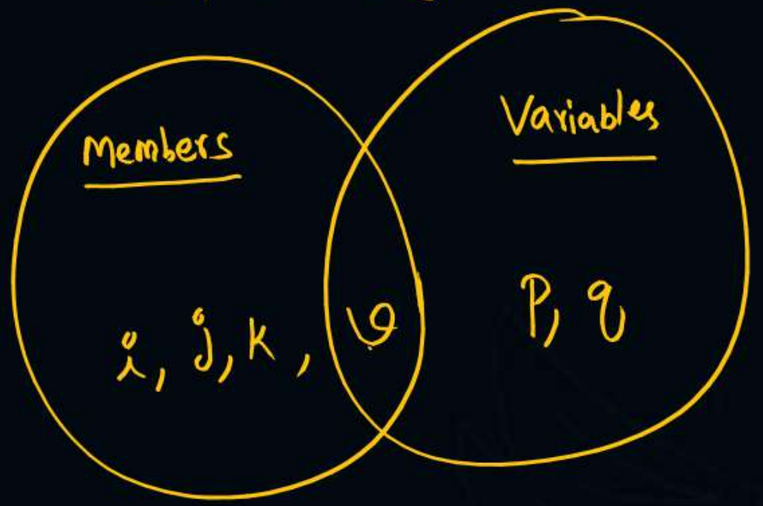
define s to be

A. An array, each element of which is a pointer to a structure of type node
B. A structure of 2 fields, each field being a pointer to an array of 10 elements
C. A structure of 3 fields: an integer, a float, and an array of 10 elements
D. An array, each element of which is a structure of type node

2 mins Summary

Self-Referential Structure (SRS)

- A Structure, whose member/variable is variable/member of itself respectively.



Ex:

```
Struct ABC
{   int i;
    float j;
    char k;
    struct ABC *q;
}*P,*q;
```

- NOTE: SRS variables must be Pointer Variables
  SRS is Used in Linked List Implementation.

THANK - YOU